- *Structure*—The `ActorSystem` hierarchy of actor references makes it possible to replace actor instances without other actors being affected.
- *Redundancy*—An actor can be replaced by another. In the example of the broken database connection, the fresh actor instance can connect to a different database. The supervisor can also decide to stop the faulty actor and create another type instead. Another option would be to route messages in a load-balanced fashion to many actors, as discussed in chapter 6.
- *Replacement*—An actor can always be re-created from its `Behavior`. A supervisor can decide to replace a faulty actor instance with a fresh one without knowing any details about re-creating the actor.
- *Reboot*—This can be done through a restart.
- *Component lifecycle*—An actor is an active component. It can be started, stopped, and restarted. The next section examines how the actor goes through its lifecycle.
- *Suspend*—When an actor crashes, its mailbox is suspended until the supervisor decides what should happen with the actor.
- *Separation of concerns*—The normal actor message-processing and supervision fault-recovery flows are orthogonal. They can be defined and evolve completely independently of each other.

Remember that the most dangerous actors (actors most likely to crash) should be as far down the hierarchy as possible. Faults that occur far down the hierarchy can be handled or monitored by more actors than faults that occur higher up the hierarchy. If a fault occurs at the top level of the `ActorSystem`, it can lead to a restart of all